

Questão 02 - Script de backup do Ledger

Prompt (aplicando R-T-F)

[ROLE]

Você é um Engenheiro DevOps/SRE Sênior, especialista em automação de backups de bancos PostgreSQL em ambientes AWS, com foco em scripts bash robustos, seguros e prontos para rodar via cron em produção.

[TASK]

Crie um script bash de backup automatizado para o banco PostgreSQL "Ledger", que será executado diariamente via cron em uma instância EC2 Ubuntu 22.04 LTS.

Ambiente e parâmetros:

- Host: ledger-db.internal.hvt.io | Porta: 5432 | Banco: ledger_prod
- Usuário: backup_user
- Senha: já disponível na variável de ambiente PGPASSWORD (populada pelo AWS Secrets Manager via IAM role da instância — o script NÃO deve buscar nem expor a senha diretamente)
- Região AWS: us-east-1
- Diretório de trabalho local: /var/backups/ledger (80 GB livres, dump compactado atual ~12 GB — considere isso ao decidir sobre limpeza local)
- Bucket S3 de destino: hvt-ledger-backups

O script deve, nesta ordem:

1. Gerar o dump com pg_dump, nomeando o arquivo com timestamp (ex: ledger_prod_YYYYMMDD_HHMMSS.sql)
2. Compactar o dump com gzip
3. Enviar o arquivo compactado ao bucket S3 via aws s3 cp
4. Aplicar retenção de 30 dias no S3, removendo objetos mais antigos que isso
5. Também limpar arquivos locais antigos em /var/backups/ledger para não estourar o espaço em disco (considerando que cada dump ocupa ~12 GB e há

apenas 80 GB livres)

6. Registrar cada execução (início, sucesso, falha, tamanho do arquivo, duração) em /var/log/ledger-backup.log com timestamp
7. Encerrar com exit code 0 em sucesso e código diferente de zero em qualquer etapa que falhar, interrompendo as etapas seguintes (fail-fast)

O script deve validar pré-requisitos (existência de pg_dump, aws cli, espaço em disco disponível) antes de iniciar o dump.

[FORMAT]

Entregue apenas o script bash completo, em um único bloco de código, com:

- Shebang e "set -euo pipefail" no topo
- Comentários explicando cada bloco funcional
- Nome de arquivo sugerido para o script na primeira linha como comentário

Não inclua explicações fora do bloco de código.

Modelo de IA utilizado

Claude Sonnet 4.5 (Anthropic)

Justificativa do uso

Modelo de referência do curso. Forte aderência a prompts estruturados em múltiplos blocos, raciocínio encadeado consistente e geração de artefatos técnicos (YAML, HCL, bash, SQL) com alto grau de precisão. Escolhido como baseline por permitir avaliação consistente dos frameworks ao longo de todo o desafio.

Output gerado

```
#!/usr/bin/env bash
```

```
# Arquivo sugerido: /opt/scripts/ledger_backup.sh
```

```
set -euo pipefail
```

```
# ----- Configurações -----
```

```
DB_HOST="ledger-db.internal.hvt.io"
DB_PORT="5432"
DB_NAME="ledger_prod"
DB_USER="backup_user"
AWS_REGION="us-east-1"
S3_BUCKET="hvt-ledger-backups"
BACKUP_DIR="/var/backups/ledger"
LOG_FILE="/var/log/ledger-backup.log"
RETENTION_DAYS=30
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
DUMP_FILE="${BACKUP_DIR}/ledger_prod_${TIMESTAMP}.sql"
GZIP_FILE="${DUMP_FILE}.gz"
MIN_FREE_GB=15 # margem de segurança acima do tamanho médio do dump (~12GB)

# ----- Função de log -----
log() {
    echo "$(date +"%Y-%m-%d %H:%M:%S") [$1] $2" >> "$LOG_FILE"
}

# ----- Tratamento de erro (fail-fast) -----
trap 'log "ERROR" "Falha na linha $LINENO. Abortando execução."; exit 1' ERR

START_TIME=$(date +%s)
log "INFO" "Iniciando rotina de backup do Ledger."

# ----- Validação de pré-requisitos -----
command -v pg_dump >/dev/null 2>&1 || { log "ERROR" "pg_dump não encontrado."; exit 2; }
command -v aws >/dev/null 2>&1 || { log "ERROR" "aws cli não encontrado."; exit 2; }
command -v gzip >/dev/null 2>&1 || { log "ERROR" "gzip não encontrado."; exit 2; }

if [ -z "${PGPASSWORD:-}" ]; then
```

```

log "ERROR" "Variável PGPASSWORD não está definida no ambiente."

exit 3

fi

mkdir -p "$BACKUP_DIR"

FREE_SPACE_GB=$(df --output=avail -BG "$BACKUP_DIR" | tail -1 | tr -d 'G ')

if [ "$FREE_SPACE_GB" -lt "$MIN_FREE_GB" ]; then

    log "ERROR" "Espaço em disco insuficiente (${FREE_SPACE_GB}GB livres, mínimo
    ${MIN_FREE_GB}GB)."

    exit 4

fi

# ----- Dump do banco -----

log "INFO" "Executando pg_dump em ${DB_NAME}@${DB_HOST}:${DB_PORT}."

pg_dump -h "$DB_HOST" -p "$DB_PORT" -U "$DB_USER" -d "$DB_NAME" -F p -f "$DUMP_FILE"

# ----- Compactação -----

log "INFO" "Compactando dump com gzip."

gzip -f "$DUMP_FILE"

DUMP_SIZE=$(du -h "$GZIP_FILE" | cut -f1)

log "INFO" "Dump gerado: ${GZIP_FILE} (${DUMP_SIZE})"

# ----- Upload para S3 -----

log "INFO" "Enviando backup para s3://${S3_BUCKET}/."

aws s3 cp "$GZIP_FILE" "s3://${S3_BUCKET}/" --region "$AWS_REGION"

log "INFO" "Upload concluído com sucesso."

# ----- Retenção no S3 (30 dias) -----

log "INFO" "Aplicando política de retenção de ${RETENTION_DAYS} dias no S3."

```

```
CUTOFF_DATE=$(date -d "-${RETENTION_DAYS} days" +%Y-%m-%d)
```

```
aws s3api list-objects-v2 --bucket "${S3_BUCKET}" --region "${AWS_REGION}" \
  --query "Contents[?LastModified<='${CUTOFF_DATE}'].Key" --output text | \
```

```
while read -r KEY; do
```

```
  if [ -n "$KEY" ]; then
```

```
    aws s3 rm "s3://${S3_BUCKET}/${KEY}" --region "${AWS_REGION}"
```

```
    log "INFO" "Removido do S3 (retenção): ${KEY}"
```

```
  fi
```

```
done
```

```
# ----- Limpeza local -----
```

```
log "INFO" "Removendo backups locais com mais de ${RETENTION_DAYS} dias."
```

```
find "${BACKUP_DIR}" -name "ledger_prod_*.sql.gz" -mtime +${RETENTION_DAYS} -exec rm -f {} \;
```

```
END_TIME=$(date +%s)
```

```
DURATION=$((END_TIME - START_TIME))
```

```
log "SUCCESS" "Backup concluído com sucesso em ${DURATION}s. Arquivo: ${GZIP_FILE} (${DUMP_SIZE})."
```

```
exit 0
```

Justificativa — como R-T-F aparece no prompt

Componente	Onde aparece	Por quê
Role	"Engenheiro DevOps/SRE Sênior, especialista em automação de backups PostgreSQL em AWS"	Situa a IA no nível de rigor esperado para script de produção (tratamento de erro, fail-fast, segurança de credenciais), diferente de um script "de tutorial"
Task	Bloco com todos os parâmetros do ambiente + as 7 etapas ordenadas + validação de pré-requisitos	Traduz exatamente as exigências do enunciado (host, banco, retenção de 30 dias, log com timestamp, exit code) em instruções acionáveis, deixando claro o que o script deve fazer e em que ordem . OBS: Um detalhe que adicionei por conta própria (fora do enunciado, mas coerente com o cenário: 80GB livres / 12GB por dump) foi a limpeza local dos dumps antigos — sem isso, em poucos dias o disco estouraria
Format	"único bloco de código, com shebang, set -euo pipefail, comentários por bloco, nome de arquivo sugerido"	Define o padrão de entrega técnica esperado por um time de SRE: script pronto para produção, sem texto solto, seguindo convenções de bash defensivo